1. Describe an experience analyzing and resolving a technical problem and your interaction with the customer/user experiencing the problem.

My neighbor Jason wanted to upgrade his home internet to a 2.5 gigabits per second (Gbps) plan. Before he paid for it, I looked at his actual equipment and told him the plan alone would not get him faster speeds, because his hardware could not carry it. His cabling was Category 5 (Cat5), he had no 2.5 Gbps network switch, and the network card built into his motherboard was limited to 1 Gbps. I explained the chain in plain terms so he understood the bottleneck was the gear and not the plan, and I recommended the specific hardware he needed, which was Category 5e (Cat5e) or better cabling, a 2.5 Gbps switch, and a 2.5 Gbps network interface card (NIC) that installs in a Peripheral Component Interconnect Express (PCIe) slot on his motherboard.

After those upgrades his speed tests still read about 1 Gbps. I traced it to the router. The unit accepted 2.5 Gbps coming in but only had 1 Gbps ports going out, so it capped everything behind it. He replaced it with a router that has 5 Gbps ports, and the tests still showed 1 Gbps. At that point I checked every connection and every setting twice and could not find anything wrong on his end. That told me the problem was upstream, so I had him contact his Internet Service Provider (ISP). The carrier had never provisioned his line for the higher tier. Once they updated it and reset the router, his tests finally showed the speed he was paying for.

Jason was happy with the wired result but asked why his WiFi was not a full 2.5 Gbps. I told him plainly that many wireless devices are not rated for that speed, that even the ones that are rarely move data fast enough to use it, and that WiFi itself varies with distance, walls, interference, and where the router sits. He understood, and he was satisfied with the outcome. What mattered most was keeping him informed at each step, setting honest expectations instead of promising a number I could not control, and working the problem in order from his cabling outward until the real cause was found.

2. What experience do you have creating documented processes and procedures, knowledge base articles, and technical documentation?

Most of my technical documentation comes from my cybersecurity internship and my own infrastructure projects, and I write it so the next person can repeat the work without me in the room. In the internship I documented vulnerability findings, the remediation steps I took, and the outcome after I re-scanned to confirm the fix held. I wrote for two readers at once, the technical reviewer who needed the exact steps and the non-technical reader who needed to know what was wrong and whether it was resolved. When I scripted a repeatable fix in PowerShell, I documented the procedure alongside it so the same change applied the same way every time.

I also documented configuration baselines. When I applied Defense Information Systems Agency Security Technical Implementation Guide (DISA STIG) hardening to Windows and Linux systems, I recorded which settings I changed and why, so the hardened state was traceable and could be checked later. One example I am proud of is a recurring high-severity finding for SQLite, which is the name of a small database program, that I documented as a false positive rather than a real exposure. It kept appearing and dropping across scans, so I searched the host for SQLite, found it was

not installed, and wrote up the evidence and my reasoning. That writeup kept the next person from chasing a fix the machine did not need.

For my infrastructure projects I keep documentation in GitHub, the version control platform, written as step-by-step setup and architecture notes. My high-availability cloud project documents the design, how to stand it up, and how the failover behaves, so the project is reproducible from the written instructions alone. I am comfortable producing the documentation this role relies on, including knowledge base articles, process and procedure writeups, and clear technical notes written to be concise for whoever reads them next.

3. Describe a project or initiative you led or supported. What was the issue or goal, what steps did you take to plan and execute the project, and what measurable results were achieved.

The project I led is a high-availability environment I designed and built on Amazon Web Services (AWS) using Terraform, an infrastructure-as-code tool that defines servers and networks in version-controlled files. The goal was a setup spread across two availability zones that stays online through a failure without paying for every managed service. I evaluated the tradeoffs and chose self-managed Network Address Translation (NAT) instances over the managed NAT Gateway service to control cost, then engineered back the reliability the managed option would have handled for me.

To plan it I mapped where a single failure could take the environment down and built for each case. The piece I am most proud of is a self-healing network route. When a NAT instance is replaced and comes back with a new network interface, an automated event rule (Amazon EventBridge) triggers a small program (AWS Lambda) that repairs the private route on its own, with no hands on the keyboard. I added a database replicated across both availability zones, access through Amazon Systems Manager (SSM) rather than Secure Shell (SSH) keys, and the built-in monitoring and audit-logging services (CloudWatch and CloudTrail). I did not want to assume it worked, so I wrote a multi-scenario failure test plan and ran it, shutting down instances across both availability zones and measuring recovery. Average recovery came in around 149 seconds with no manual intervention, and I documented the architecture and test results. The project is still a work in progress, but building it taught me that designing for availability is mostly planning for the failure you hope never comes.

A shorter example from my internship shows the same approach on a different problem. Across several remediation cycles I took a target machine from about 30 vulnerability findings down to 3 and eliminated every critical-severity finding, verifying each fix with a follow-up scan before calling it done. The common thread is the same in both. Set a clear goal, work in measured steps, and prove the result rather than assume it.